

BOBATHON 2026 · TRACK: AI

ChainSentinel

Supply Chain Disruption Assistant & Fleet Utilisation Optimizer — a Bob-powered logistics control tower that turns a declared disruption into a ranked, explainable, human-approved action plan.

Team Rangers And — Member 1 (Logistics & Optimisation) · Member 2 (Cold-Chain, AI & Bob)

Repository: github.com/Ronit069/bob-ai-hackathon-team-rangers

PROBLEM

Disruption response is blind when risk lives in three separate tools

Mid-size freight operators cannot see disruption impact, fleet idle capacity, and cold-chain integrity together.

Who experiences it

Operators running hundreds of shipments across carriers, regions, and temperature-controlled cargo — without an enterprise control tower budget.

What happens today

A port strike, weather event, or geopolitical crisis cascades across shipments; response means hours of manual cross-referencing between disconnected systems.

Why it hurts

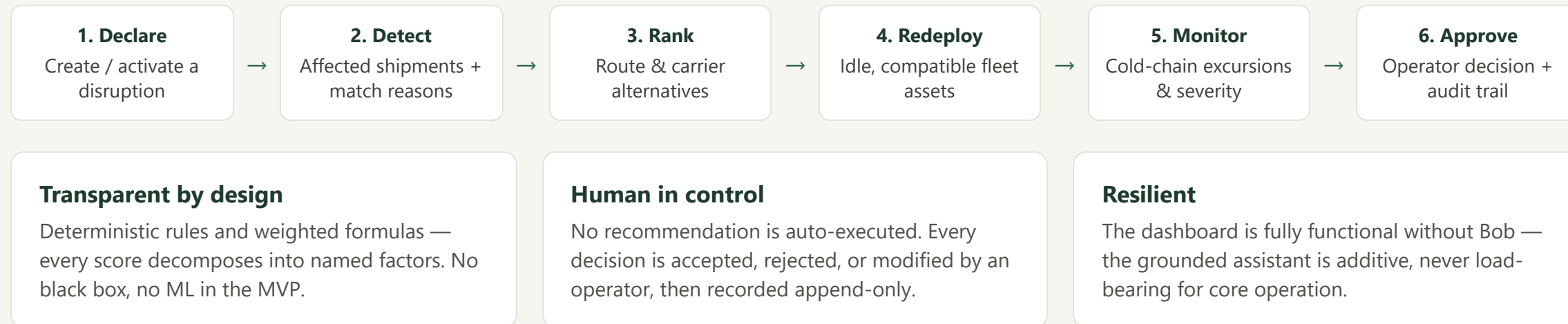
Affected shipments are found late, idle compatible assets stay idle, and temperature excursions are discovered at delivery — when the cargo is already a write-off.

Core pain: by the time a temperature excursion is found, the cargo may already be a write-off — and no one can prove after the fact why a decision was made.

SOLUTION

One ranked, explainable, human-approved action plan

ChainSentinel unifies disruption response, fleet redeployment, and cold-chain monitoring in a single dashboard.



Key features, all working end to end

Disruption management

Create, activate, resolve; real-time affected-shipment detection with match reasons and impact scores.

Ranked alternatives

Route and carrier options scored on cost, ETA, capacity, reliability; rejected options carry their failed constraint.

Fleet redeployment

Idle detection from availability and reservations, compatibility rules, ranked redeployment candidates.

Cold-chain monitoring

Sensor ingestion with data-quality flags (gaps, duplicates, out-of-order, implausible) and per-shipment sensor health.

Excursion detection

Severity ladder — Warning / Major / Critical / Unknown-Review — against configurable temperature policies, with rationale.

Combined priority

$0.5 \times \text{disruption_risk} + 0.5 \times \text{coldchain_risk}$ into a single cross-domain worklist.

Decision lifecycle

Pending → accepted / rejected / modified, with an append-only audit trail for every action.

Grounded Bob assistant

11 read-only MCP tools; raw evidence shown beside every answer; explicit fallback when unavailable.

Deterministic services, frozen data, grounded AI layer

FRONTEND

React 18 + Vite 5 dashboard

14 screens — overview worklist, disruptions, shipment detail, alternatives, fleet, cold-chain, sensor health, risk explanation, Bob chat, audit. Renders backend values verbatim; never computes scores.

BACKEND

Node.js 20 + Express 4 API

28 REST endpoints, zod validation, pg (no ORM). All formulas live in services — matching, alternatives, fleet, excursion, and combined risk.

DATA

PostgreSQL 16 + seeded fixtures

Python 3.11 generator produces frozen, deterministic fixtures: 48 shipments, 5 disruptions, 2,201 sensor readings, 48 ground-truth scenarios.

IBM TECHNOLOGY

Bob via MCP tool server

Node stdio MCP adapter exposing 11 read-only tools over the REST API. Backend proxies operator questions to Bob; answers are grounded in live tool calls with visible evidence.

Ground truth is enforced across languages: the Python oracle, REST services, MCP tools, stdio server, and smoke test must all agree on the same numbers.

Explainable scoring — every number has a reason

- **Affected shipments:** route-segment matching with timing analysis, impact score, confidence level, and explicit match reason.
- **Alternatives:** weighted factors (cost delta, ETA delta, capacity margin, carrier reliability, residual risk) plus hard constraints; rejected candidates list the constraint that failed.
- **Fleet:** idle windows from availability vs reservations, compatibility rules, ranked redeployment by distance and capacity.
- **Cold-chain:** per-cargo temperature bands, excursion grouping, duration/magnitude severity ladder, data-quality flags surfaced rather than hidden.

Combined priority score

```
combined_score = 0.5 × disruption_risk + 0.5 × coldchain_risk
```

Worked oracle — shipment S039

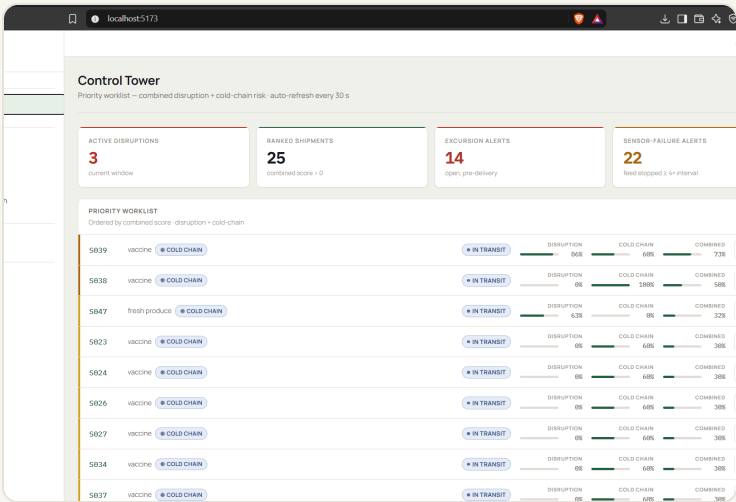
disruption 0.856 · cold-chain 0.600 · combined **0.728**

Asserted identically in backend tests, REST response, MCP tool output, stdio server, and grounding smoke — the same number everywhere.

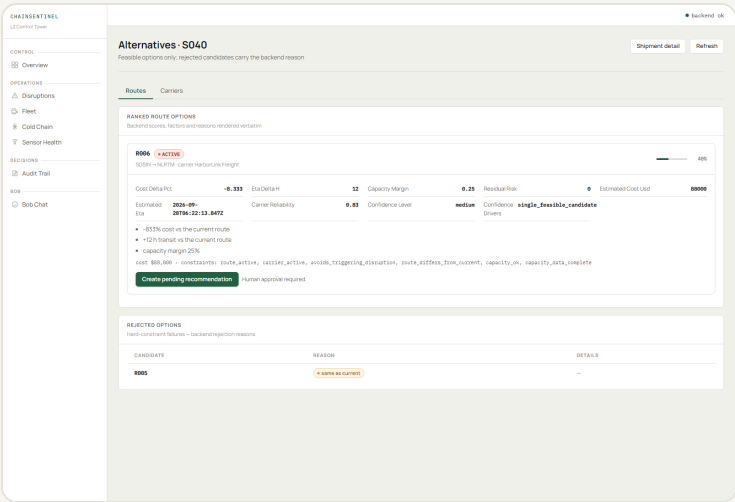
No ML in the MVP. Rules and transparent weights beat an unexplainable model for a human-approval workflow.

Live demo highlights — real data, real decisions

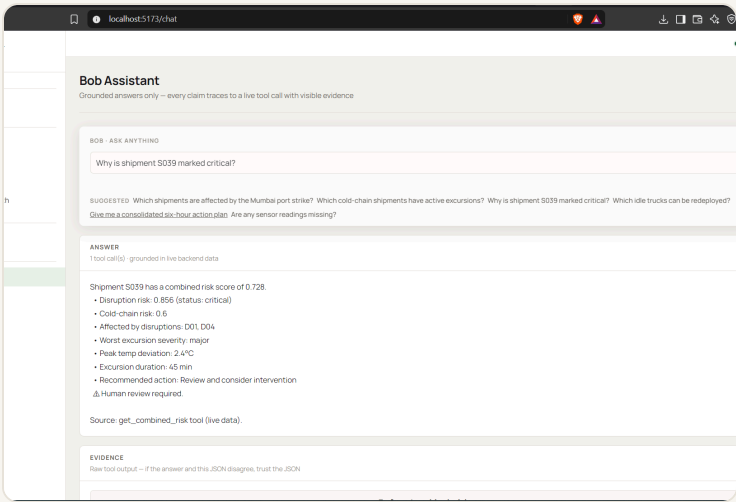
- **D01 port strike:** affected list ranks S039 critical (impact 76%), with per-shipment match reasons and deadlines.
- **Alternatives:** ranked options with full factor breakdown; rejected candidates show why (e.g. *same as current, disrupted region overlap*).
- **Cold-chain:** open excursions with severity rationale; sensor health flags data-quality issues before cargo is lost.
- **Bob:** grounded answer for S039 — one live tool call, raw evidence shown; `npm run seed` restores the exact baseline.



S1 Overview — combined risk workload



S5 Alternatives — factors & rejected options



S13 Bob — grounded answer + evidence

Bob is grounded: 11 read-only tools, zero invented data

Where Bob sits: the backend exposes `POST /api/bob/query` ; the MCP stdio server exposes the same backend data as 11 read-only tools. Bob answers only from tool output.

- `get_active_disruptions` · `get_affected_shipments`
- `get_route_alternatives` · `get_carrier_alternatives`
- `get_idle_assets` · `get_redeployment_candidates`
- `get_sensor_status` · `get_temperature_excursions`
- `get_combined_risk` · `get_risk_overview` · `get_audit_log`

Trust mechanics

Raw tool JSON is displayed beside every answer — if the answer and the evidence disagree, the operator sees it immediately. Empty or failed tools produce explicit "no data" responses, never guesses.

Graceful fallback

With `BOB_ENABLED=false` or without credentials, the chat panel shows a clear fallback state; the rest of the dashboard is unaffected.

Provable grounding

`npm run smoke` — 7/7 checks from the live backend, including the S039 = 0.728 oracle through the MCP layer.

Engineering evidence you can run yourself

147

backend tests passing (Node test runner)

49

frontend tests across 14 screens

48

Python generator tests + ground-truth scenarios

7/7

MCP grounding smoke checks on live data

Cross-language truth

verify-seed: matching 15/15 and excursion 14/14 — the JS services must agree with the Python oracle, reproducible from a fixed seed.

Frozen contracts

API and data contracts frozen in docs; append-only audit; a static scan test enforces that no formula ever lives in the frontend.

Full state coverage

Every screen handles loading, empty, validation error, unexpected error, Bob 503 fallback, unknown/review, not-actionable, and conflict states.

Reproducibility is the demo: one seed, one command, and every score in the UI is verifiable against the frozen fixtures.

From hackathon prototype to operating capability

Immediate value

Replace hours of manual cross-referencing with a ranked worklist and explainable options — disruption triage in minutes, with an auditable decision record.

Prevented losses

Cold-chain excursions surfaced in-transit instead of at delivery: fewer write-offs, faster intervention, better fleet utilisation from redeployment of idle compatible assets.

Clear integration path

Live carrier APIs, weather/port feeds, and MQTT/Kafka sensor streaming slot behind the existing ingestion and REST contracts — no redesign needed.

Intelligence roadmap

ML ETA-delay prediction and sensor-stream anomaly detection are the explicitly deferred next step — built on top of a transparent deterministic baseline, not replacing it.

Governance-ready: append-only audit and human approval mean every future recommendation is explainable after the fact — a requirement for regulated supply-chain operations.

What this is — and what it is not

Synthetic data only

No real shipment, carrier, sensor, or regulatory dataset is used or claimed. Fixtures are deterministic and reproducible.

No ML in the MVP

All logic is deterministic rules and transparent weighted scoring. ML is explicitly deferred, not camouflaged.

Illustrative thresholds

Temperature policy values are configurable placeholders, not regulatory benchmarks. Real deployments need product- and jurisdiction-specific rules.

Local-only, single operator

No cloud deployment, no authentication or multi-tenant isolation. Bob grounding reduces but does not eliminate hallucination risk.

THANK YOU

ChainSentinel — decisions you can defend

- **Built end to end:** dashboard, API, database, deterministic data generator, and the IBM Bob MCP layer — with 246 automated checks behind them.
- **Grounded by construction:** Bob never invents data; evidence is always on screen.
- **Human-approved by design:** every recommendation needs an operator decision, recorded append-only.

Team Rangers And — Member 1 (Logistics & Optimisation) · Member 2 (Cold-Chain, AI & Bob)

Repository: github.com/Ronit069/bob-ai-hackathon-team-rangers · Docs: [docs/architecture.md](#), [docs/setup-guide.md](#)